1 es安装
1.1 简介
1.1.1 es是什么
1.1.2 elk是什么
1.1.3 索引是什么?
1.1.4 文档是什么?
1.1.5 字段是什么?
1.1.6 映射是什么?
1.1.7 分片是什么
1.1.8 副分片
1.1.9 DSL是什么?
1.1.10 集群和节点cluster&node
1.2 安装
1.2.1 下载镜像
1.2.2 创建
1.3 账密（windows本机）
1.4 账密（mac）
2 基础操作
2.1 索引操作
2.1.1 创建索引
2.1.2 关闭索引
2.1.3 打开索引
2.1.4 删除索引
2.1.5 索引别名
2.2 映射操作
2.2.1 查看映射
2.2.2 扩展映射
2.2.3 基本数据类型
2.2.4 复杂数据类型
2.3 文档操作
2.3.1 单条写入
2.3.2 批量写入
2.3.3 更新单条文档
2.3.4 批量更新文档
2.3.5 根据条件更新文档
2.3.6 删除单条文档
2.3.7 批量删除文档
2.3.8 根据条件删除文档
3 es python api
3.1 python连接
3.2 操作
3.2.1 创建索引
3.2.2 插入数据
3.2.3 查看索引中的文档数量
3.2.4 查询所有

3.2.5 根据term查询
3.2.6 根据 terms 查询
3.2.7 range
3.2.8 exists / missing
3.2.9 bool过滤
3.2.10 multi_match
3.2.11 regexp 查询
3.2.12 prefix 查询
3.2.13 phrase match 短语匹配
3.2.14 根据id删除数据
3.2.15 delete_by_query
4 案例
4.1 python+BERT+es(英文)
4.1.1 简介
4.1.2 代码

1 es安装

1.1 简介

1.1.1 es是什么

- 建立在Lucene基础之上的分布式准实时搜索引擎

  lucene是一个老牌的搜索技术支持库，提供的很多功能都能用于处理文本类型的数据。然而，lucene仅仅提供了基础的搜索引擎支持，而对搜索的分布式、容错性和实时性并不支持

1.1.2 elk是什么

- E代表 `Elasticsearch` , 主要提供数据搜索和分析功能;
- L代表 `Logstash` , 借助它可以将数据库和日志等结构化或非结构化数据轻松地导人ES中; |
- K代表 `Kibana` , 它可以将分析结果进行图形化展示, 此外还可以使用它提供的开发工具对ES进行请求的交互。

1.1.3 索引是什么?

- 在使用传统的关系型数据库时, 如果对数据有存取和更新操作,需要建立一个数据库。相应地，在ES中则需要建立索引。【可以理解为： ES是一种数据库, 索引是库里的数据表。】用户的数据新增、搜索和更新等操作的对象全部对应索引。

1.1.4 文档是什么?

- 在使用传统的关系型数据库时, 需要把数据封装成数据库中的一条记录,而在ES中对应的则是文档。
- ES的文档中可以有一个或多个字段, 每个字段可以是各种类型。用户对数据操作的最细粒度对象就是文档。

1.1.5 字段是什么?

- 一个文档可以包含一个或多个字段, 每个字段都有一个类型与其对应。
- 除了常用的数据类型（如字符串型、文本型和数值型）外，ES还提供了多种数据类型, 如数组类型、经纬度类型和IP地址类型等。
- ES对不同类型的字段可以支持不同的搜索功能。

1.1.6 映射是什么?

- 建立索引时需要定义文档的数据结构, 这种结构叫作映射。在映射中, 文档的字段类型一旦设定后就不能更改。因为字段类型在定义后, ES已经针对定义的类型建立了特定的索引结构,这种结构不能更改。借助映射可以给文档新增字段。
- ES还提供了自动映射功能, 即在添加数据时, 如果该字段没有定义类型, ES会根据用户提供

的该字段的真实数据来猜测可能的类型, 从而自动进行字段类型的定义。

1.1.7 分片是什么

- 在分布式系统中，为了能存储和计算海量的数据，会先对数据进行切分，然后再将他们存储到多台计算机中。
- 在ES中，一个分片对应的就是一个Lucene索引，每个分片可以设置多个副分片，这样当主分片所在的计算机因为发生故障而离线时，副分片会充当:主分片继续服务。
- 索引的分片个数只能设置一次，之后不能更改。在默认情况下，ES的每个索引设置为5个分片
- 可以理解为多个备份

1.1.8 副分片

- 为了提升系统索引数据的高可用性并减轻集群搜索的负载, 可以启用分片的副本，该副本叫作副分片，而原有分片叫作主分片。
- 在默认情况下, ES不会为索引的分片开启副分片, 用户需要手动设置。副分片的个数设定后，也可以进行更改。
- 一个分片的主分片和副分片分别存储在不同的计算机上，通过这种分布式结构提升了分片数据的高可用性和服务的高并发支持。

1.1.9 DSL是什么?

- ES使用`DSL（Domain Specific Language，领域特定语言）`来定义查询。与编程语言不同, DSL是在特定领域解决特定任务的语言, 它可以有多种表达形式, 如我们常见的HTML、CSS、SQL等都属于DSL。
- ES中的DSL采用JSON进行表达，相应地，ES也将响应客户端请求的返回数据封装成了JSON形式。这样不仅可以简单明了地表达请求/响应内容, 而且还屏蔽了各种编程语言之间数据通信的差异。

1.1.10 集群和节点cluster&node

- 集群由一个唯一的名字表示，默认为Elasticsearch。

  集群抿成非常重要，具有相同集群抿成的节点才会组成一个集群。集群抿成可以在配置文件中指定
- Elasticsearch中的一个节点也被称为集群。
- ES集群的节点个数不受限制，用户可以根据需求增加计算机对搜索服务进行扩展。

1.2 安装

- 先为elasticsearch和kibana创建docker network

  > docker network create elastic

1.2.1 下载镜像

- `elasticsearch` 镜像

> docker pull elasticsearch:8.15.2

- `kibana` 镜像

> docker pull kibana:8.15.2

- `logstash` 镜像

1.2.2 创建

- 创建单节点的ES

> docker run --name es01 --net elastic -p 9200:9200 -p 9300:9300 -it elasticsearch:8.15.2

`注意`:运行命令后，最后输出的信息包含非常重要的密码和秘钥

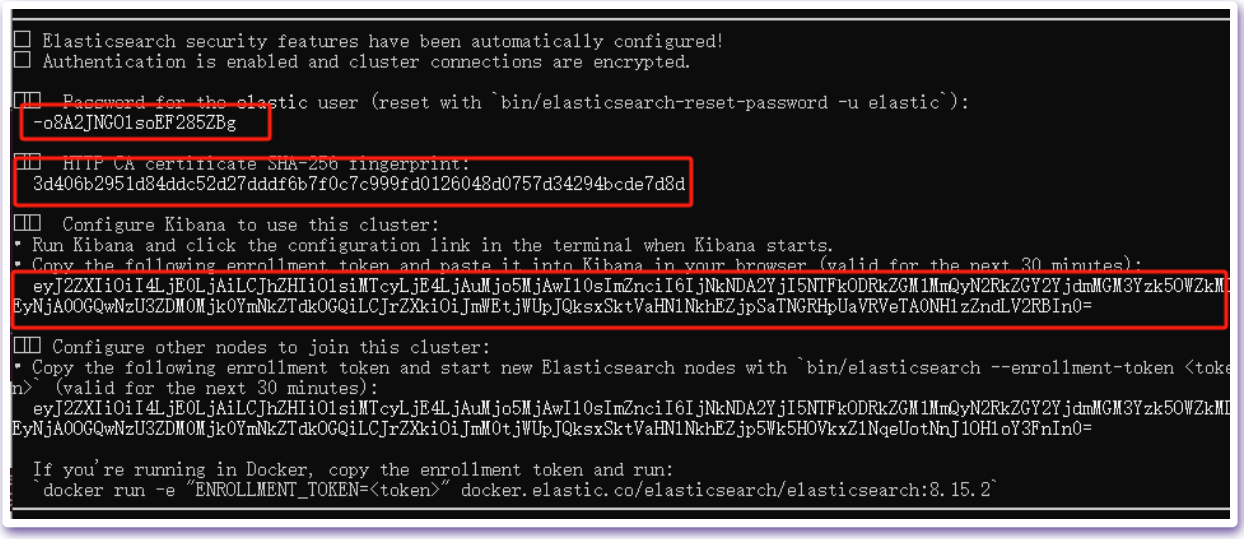
window中

```
━━━
```

```
• Copy the following enrollment token and start new Elasticsearch nodes with `bin/elasticsearch --enrollment-token <token>` (valid for the next 30 minutes):

 eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE4LjAuMjo5MjAwIl0sImZnciI6IjkwNzlkYzJiNzUzYWZhMDRlZjYwMWZiMjM5ZjJkM2IyYTYyMzljOWE5ODY1MDRkZjgzOGU0Y2M2NGJjMzc3OTYiLCJrZXkiOiJHQVh5OFpJQmlBcm1aZklPdDktdjphNFc0NzR2YVMybTc1Vmt1aHJUaDN3In0=

  If you're running in Docker, copy the enrollment token and run:
  `docker run -e "ENROLLMENT_TOKEN=<token>" docker.elastic.co/elasticsearch/elasticsearch:8.15.2`
```



mac中

```
✅ Elasticsearch security features have been automatically configured!
✅ Authentication is enabled and cluster connections are encrypted.

ℹ️  Password for the elastic user (reset with `bin/elasticsearch-reset-password -u elastic`):
  Wp9h0OlmhPO5uI-3z1nM

ℹ️  HTTP CA certificate SHA-256 fingerprint:
  2ca4abf6522b256743f4f5261df4fd5e452cb7ed5760d62589d9a5112321afb0

ℹ️  Configure Kibana to use this cluster:
• Run Kibana and click the configuration link in the terminal when Kibana starts.
• Copy the following enrollment token and paste it into Kibana in your browser (valid for the next 30 minutes):
```

```
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE5LjAuMjo5MjAwIl0sImZnciI6IjJ
jYTRhYmY2NTIyYjI1Njc0M2Y0ZjUyNjFkZjRmZDVlNDUyY2I3ZWQ1NzYwZDYyNTg5ZD
lhNTExMjMyMWFmYjAiLCJrZXkiOiI3QVV1ajVJQnFXNlBQdlN0d25XRDozeVFnVDV2W
VFwcTlsc0VZMUw3aFBnIn0=

ℹ️ Configure other nodes to join this cluster:
• Copy the following enrollment token and start new Elasticsearch
nodes with `bin/elasticsearch --enrollment-token <token>` (valid
for the next 30 minutes):

eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE5LjAuMjo5MjAwIl0sImZnciI6IjJ
jYTRhYmY2NTIyYjI1Njc0M2Y0ZjUyNjFkZjRmZDVlNDUyY2I3ZWQ1NzYwZDYyNTg5ZD
lhNTExMjMyMWFmYjAiLCJrZXkiOiI3Z1V1ajVJQnFXNlBQdlN0d25XRDpfWjl4U3pGU
lNLdTMtaTBBYkRYejV3In0=

  If you're running in Docker, copy the enrollment token and run:
  `docker run -e "ENROLLMENT_TOKEN=<token>"
docker.elastic.co/elasticsearch/elasticsearch:8.15.2`
━━━
```

- 账密

> 账号：elastic
>
> 密码：sHZL+UMbav+xzdSzDw0Y

- http fingerprinter

> 9079dc2b753afa04ef601fb239f2d3b2a6239c9a986504df838e4cc64bc37796

- 密钥

> eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE4LjAuMjo5MjAwIl0sImZnciI6IjkwNzlkYzJiNzUzY
> WZhMDRlZjYwMWZiMjM5ZjJkM2IyYTYyMzljOWE5ODY1MDRkZjgzOGU0Y2M2NGJjMzc3OTYi
> LCJrZXkiOiJHQVh5OFpJQmlBcm1aZklPdDktdjphNFc0NzR2YVMybTc1Vmt1aHJUaDN3In0=

1.4 账密（mac）

- 账密

> 账号：elastic
>
> 密码：H40Usw1A*RvC3h_2F50j

- http fingerprinter

> 50733eba945ee86d1913507d5847d4f2b0f3aaef1d0d9d832cb017d5bab0aee4

- 密钥

> eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiMTcyLjE5LjAuMjo5MjAwIl0sImZnciI6IjUwNzMzZWJhOTQ
> 1ZWU4NmQxOTEzNTA3ZDU4NDdkNGYyYjBmM2FhZWYxZDBkOWQ4MzJjYjAxN2Q1YmFiM
> GFlZTQiLCJrZXkiOiJpalZ0aHBJQmwwTkRwdEI4UkxkSjoyanpNU2RLUVRocVlieFhBOUtXeV
> lnIn0=

2 基础操作

- 查看本机elasticsearch集群的健康状态

```
GET _cluster/health
```

2.1 索引操作

2.1.1 创建索引

- 创建名称为person的索引，

```
PUT /person
{
  "settings":{
    "number_of_shards": 1, // 指定主分片个数,
    "number_of_replicas": 1 // 指定副分片个数
  },
  "mappings":{
    "properties": {
      "name":{ // 属性1
        "type":"text"
      },
      "age":{ // 属性2
        "type":"integer"
      }
    }
  }
}
```

2.1.2 关闭索引

- 关闭索引

```
POST /person/_close
```

2.1.3 打开索引

- 打开索引

```
POST /person/_open
```

2.1.4 删除索引

- 删除索引

```
DELETE /person
```

2.1.5 索引别名

- 别名是指给一个或者多个索引定义另外一个名称，使索引别名和索引之间可以建立某种逻辑关系

  以下将名为book的index起别名为book_alias

```
POST /_aliases
{
  "actions":[
    {
      "add":{
        "index":"book",
        "alias":"book_alias"
      }
    }
  ]
}
```

  以下将名为book的index别名为删除

```
POST /_aliases
{
  "actions":[
    {
      "remove":{ // 删除原先的索引别名
        "index":"book",
        "alias":"book_alias"
      }
    }
  ]
}
```

- 示例

```
// ① 创建索引
PUT /hotel
{
  "settings":{
    "number_of_shards": 1,  // 主分片
    "number_of_replicas": 2 // 副分片
  },
  "mappings":{
    "properties":{
      "title":{ // 属性1
        "type":"text"
      },
      "city":{ // 属性2
        "type":"keyword"
      },
      "price":{ // 属性3
        "type":"double"
```

```
        }
      }
    }
  }

  // ② 向索引hotel中写入1条数据
  POST /hotel/_doc/001
  {
    "title":"喜来登酒店",
    "city":"深圳",
    "price":1099.00
  }

  // ③ 建立别名 guesthouse
  POST /_aliases
  {
    "actions":[
      {
        "add":{
          "index":"hotel",
          "alias":"guesthouse"
        }
      }
    ]
  }

  // ④ 使用别名guesthouse进行搜索
  //  (1) 使用hotel
  POST /hotel/_search
  {
    "query":{
      "match":{
        "title":"喜来登"
      }
    }
  }

  //  (2) 使用guesthouse
  POST /guesthouse/_search
  {
    "query":{
      "match":{
        "title":"喜来登"
      }
    }
  }

  // ⑤ 新的需求，设置主分片数为2个，副分片数为4个，创建新的索引 hotelv2
  PUT /hotelv2
  {
    "settings":{
      "number_of_shards":2,
```

```
      "number_of_replicas":4
    },
    "mappings":{
      "properties":{
        "title":{
          "type":"text"
        },
        "city":{
          "type":"keyword"
        },
        "price":{
          "type":"double"
        }
      }
    }
  }

  // ⑥ 向新的索引hotelv2中写入数据
  POST /hotelv2/_doc/001
  {
    "title":"万豪酒店",
    "city":"深圳",
    "price":999.99
  }

  // ⑦ 现在删除hotel的索引别名，设置索引hotelv2的别名为guesthouse
  POST /_aliases
  {
    "actions":[
      {
        "remove":{ // 删除原先的索引别名
          "index":"hotel",
          "alias":"guesthouse"
        }
      },
      {
        "add":{ // 新建索引别名
          "index":"hotelv2",
          "alias":"guesthouse"
        }
      }
    ]
  }

  // ⑧ 使用索引别名guesthouse检索
  POST /guesthouse/_search
  {
    "query":{
      "match":{
        "title":"万豪"
      }
    }
```

```
}
```

2.2 映射操作

2.2.1 查看映射

- 查看映射

```
GET /hotel/_mapping
```

2.2.2 扩展映射

- 扩展映射

```
POST /hotel/_mapping
{
  "properties":{
    "address":{ // 扩展hotel索引，并增加address字段
      "type":"text"
    }
  }
}
```

2.2.3 基本数据类型

2.2.3.1 keyword（不可切分，只能全词语匹配）

- 创建索引 user

```
PUT /user
{
  "mappings":{
    "properties":{
      "name":{
        "type":"keyword"
      }
    }
  }
}
```

- 写入1条数据

```
PUT /user/_doc/001
{
  "name":"周杰伦"
}

```

- 使用term进行全字符串匹配（可以查到）

```
GET /user/_search
{
  "query":{
    "term":{
      "name":"周杰伦"
    }
  }
}
```

- 使用match搜索"杰伦",无法命中文档,返回空（无法查到）

```
GET /user/_search
{
  "query":{
    "match":{
      "name":"杰伦"
    }
  }
}
```

- 使用match搜索"周杰伦",可以命中

```
GET /user/_search
{
  "query":{
    "match":{
      "name":"周杰伦"
    }
  }
}
```

2.2.3.2 text类型

- 创建索引car

```
PUT /car
{
  "mappings":{
    "properties": {
      "title":{
```

```
          "type":"text"
        },
        "brand":{
          "type":"keyword"
        },
        "price":{
          "type":"float"
        }
      }
    }
  }
```

- 写入数据

```
POST /car/_doc/001
{
  "title":"宝马 x7",
  "brand":"BMW",
  "price":1030000
}
```

- 使用match检索

```
GET /car/_search
{
  "query":{
    "match":{
      "title":"x7"
    }
  }
}
```

- 使用term检索

`term搜索用于搜索值和文档对应的字段是否完全相等`，而对于text类型的数据，在建立索引时ES已经进行了切分并建立了倒排索引，因此使用term `没有搜索到` 数据。

```
GET /car/_search
{
  "query":{
    "term":{
      "title":"宝马 x7"
    }
  }
}
```

使用match可以搜索到

```
GET /car/_search
{
  "query":{
    "match":{
      "title":"宝马 x7"
    }
  }
}
```

2.2.3.3 数值类型

- ES支持的数值类型有long、integer、short、byte、double、float、half_float、scaled_float和unsigned_long等
- 创建索引

```
PUT /apartment
{
  "mappings": {
    "properties": {
        "title":{
        "type":"text"
        },
        "city":{
          "type":"keyword"
        },
        "building year":{
          "type":"integer"
        }
    }
  }
}
```

- 写入数据

```
POST /apartment/_doc/001
{
  "title":"深圳湾壹号",
  "city":"深圳",
  "building year":"2019"
}

POST /apartment/_doc/001
{
  "title":"深圳湾壹号",
  "city":"深圳",
  "building year":"2019"
}

```

```
POST /apartment/_doc/002
{
  "title":"汉森吉祥龙",
  "city":"深圳",
  "building year":"2022"
}

POST /apartment/_doc/003
{
  "title":"珠光花半里",
  "city":"深圳",
  "building year":"2007"
}
```

- 检索数据

  building year 大于等于2006，小于等于2021

  1. `lt` ： less than 小于
  2. `lte`： less than equal 小于等于
  3. `gt`： grater than 大于
  4. `gte`： grater than equal 大于等于

```
GET /apartment/_search
{
  "query":{
    "range":{
      "building year":{
        "gte":2006,
        "lte":2021
      }
    }
  }
}
```

2.2.3.4 布尔类型

- 创建索引

```
PUT /player
{
  "mappings":{
    "properties":{
      "name":{
        "type":"text"
      },
      "age_over_tweenty":{
        "type":"boolean"
      }
    }
  }
}
```

- 写入数据

```
POST /player/_doc/001
{
  "name":"John",
  "age_over_tweenty":"true"
}

POST /player/_doc/002
{
  "name":"Sally",
  "age_over_tweenty":"true"
}

POST /player/_doc/003
{
  "name":"Jason",
  "age_over_tweenty":"false"
}
```

- 使用boolean类型检索

```
GET /player/_search
{
  "query":{
    "match":{
      "age_over_tweenty": true
    }
  }
}
```

2.2.3.5 日期类型-date

- 在ES中，日期类型的名称为date。ES中存储的日期是标准的UTC格式
- 创建索引student

```
PUT /student
{
  "mappings":{
    "properties": {
      "name":{
        "type":"text"
      },
      "birthday":{
        "type":"date"
      }
    }
  }
}
```

- 写入数据

```
POST /student/_doc/001
{
  "name":"张三",
  "birthday":"2000-01-23"
}

POST /student/_doc/002
{
  "name":"李四",
  "birthday":"1998-05-16"
}

POST /student/_doc/003
{
  "name":"王五",
  "birthday":"2005-08-08T22:15:45"
}
```

- 检索数据 birthday

```
GET /student/_search
{
  "query":{
    "range":{
      "birthday": {
        "gte": "2000-01-01",
        "lte": "2006-01-01"
      }
    }
  }
}
```

2.2.3.6 日期类型-自定义format

- 日期类型默认不支持 **yyyy-MM-dd HH:mm:ss** 格式，如果经常使用这种格式，可以在索引的mapping中设置日期字段的 `format` 属性为自定义格式。
- 创建索引示例

```
PUT /student_v2
{
  "mappings":{
    "properties": {
      "name":{
        "type":"text"
      },
      "birthday":{
        "type":"date",
        "format":"yyyy-MM-dd HH:mm:ss"  // 自定义格式
      }
    }
  }
}
```

2.2.4 复杂数据类型

2.2.4.1 数组类型

- ES数组没有定义方式，其使用方式是开箱即用的，即无须事先声明，在写入时把数据用中括号[]括起来，由ES对该字段完成定义。

```
// ① 创建索引
PUT /type_v1
{
  "mappings":{
    "properties": {
      "name":{
        "type":"text"
      }
    }
  }
}

// ② 写入数据
POST /type_v1/_doc
{
  "name":"Simon",
  "hobbies":["打篮球","听音乐","看电影","狼人杀"]
}

// ③ 查询映射
GET /type_v1/_mapping

```

映射如下

```
{
  "type_v1": {
    "mappings": {
      "properties": {
        "hobbies": {
          "type": "text",
          "fields": {
            "keyword": {
              "type": "keyword",
              "ignore_above": 256
            }
          }
        },
        "name": {
          "type": "text"
        }
      }
    }
  }
}
```

- 如果事先已经定义了字段类型（`必须为keyword`），在写数据时以数组形式写入，ES也会将该类型转为数组。

```
// ① 创建索引
PUT /type_v2
{
  "mappings":{
    "properties": {
      "name":{
        "type":"text"
      },
      "hobbies":{
        "type":"keyword" // 必须设置为keyword才能检索数组中的元素
      }
    }
  }
}

// ② 写入数据
POST /type_v2/_doc
{
  "name":"Simon",
  "hobbies":["打篮球","听音乐","看电影","狼人杀"]
}

// ③ 查看映射
GET /type_v2/_mapping

```

映射如下

```
{
  "type_v2": {
    "mappings": {
      "properties": {
        "hobbies": {
          "type": "keyword"
        },
        "name": {
          "type": "text"
        }
      }
    }
  }
}
```

```

// ④ 查看数据
GET /type_v2/_search
{
  "query":{
    "match_all":{}
  }
}

// ⑤ 查询数组中的元素，字段类型必须为keyword
GET /type_v2/_search
{
  "query":{
    "term":{
      "hobbies":{
        "value":"看电影"
      }
    }
  }
}
```

2.2.4.2 对象类型

- 和数组类型一样，对象类型也不用事先定义，在写入文档的时候ES会自动识别并转换为对象类型。
- 创建索引 restaurant

```
PUT /restaurant
{
  "mappings":{
    "properties": {
```

```
        "name":{  // 餐馆名称
          "type":"text"
        },
        "food type":{ // 菜品类型
          "type":"text"
        },
        "address":{ // 位置
          "type":"text"
        }
      }
    }
  }
```

- 写入数据（增加之前没有的数据属性：comment_info餐馆评论数量）

```
POST /restaurant/_doc/001
{
  "name":"东北磨磨",
  "food type":"东北菜",
  "adress":"深圳南山区科兴科学园",
  "comment_info":{  // 评论数量
    "properties":{
      "good": 10,
      "bad": 2
    }
  }
}

POST /restaurant/_doc/002
{
  "name":"乐凯撒",
  "food type":"披萨",
  "adress":"深圳南山区科兴科学园",
  "comment_info":{  // 评论数量
    "properties":{
      "good": 30,
      "bad": 4
    }
  }
```

- 查询数据，好评数量>20

```
GET /restaurant/_search
{
  "query":{
    "range":{
      "comment_info.properties.good": {
        "gt": 20
      }
    }
  }
}
```

- 对象内部还可以包含对象，评论信息字段comment_info可以增加前3条好评数据

```
POST /restaurant/_doc/_003
{
  "name":"陶居居",
  "food type":"粤菜",
  "adress":"深圳南山区",
  "comment_info":{
    "good":50,
    "bad":6,
    "top3 good":{
      "top1":{
        "content":"就餐环境很不错，干净清爽",
        "score":92
      },
      "top2":{
        "content":"很多粤菜很好吃，口感细腻",
        "score":95
      },
      "top3":{
        "content":"服务员很专业，上菜速度很快",
        "score":86
      }
    }
  }
}
```

- 查询所有数据

```
GET /restaurant/_search
```

- 查看映射

```
GET /restaurant/_mapping
```

2.2.4.3 地理类型-geo_shape

- geo_shape（空间位置）类型支持地理形状的搜索，即支持点、线、圈、多边形搜索等。比如我们想要找到最接近给定位置的路线，就可以使用此类型。
- 创建索引 geo_shape_case

```
PUT /geo_shape_case
{
  "mappings":{
    "properties":{
      "city":{
        "type":"text"
      },
      "location":{
        "type":"geo_shape"
      }
    }
  }
}
```

- 写入数据

```
POST /geo_shape_case/_doc/001
{
  "city":"深圳",
  "location":{
    "type":"point",
    "coordinates":[18.400544, 52.530286]
  }
}
```

- 检索范围内的数据

```
GET /geo_shape_case/_search
{
  "query":{
    "bool":{
      "must":{
        "match_all":{}
      },
      "filter":{
        "geo_shape":{
          "location":{
            "shape":{
              "type":"envelope", // 包围,下面是左上角和右下角两个顶点
              "coordinates": [[13.0,53.0],
                              [14.0,52.0]]
            },
            "relation":"within"
```

```
            }
          }
        }
      }
    }
  }
```

2.2.4.4 地理类型-geo_point

- 创建索引：geo_point_case

```
PUT /geo_point_case
{
  "mappings":{
    "properties":{
      "hotel_name":{
        "type":"text"
      },
      "hotel_location":{
        "type":"geo_point"
      }
    }
  }
}
```

- 写入数据

```
POST /geo_point_case/_doc/001
{
  "hotel_name":"希尔顿酒店",
  "hotel_location":{
    "lat":40.123192,  //维度
    "lon":-71.341213 // 经度
  }
}
```

- 检索数据- 情形1 geo_bouding_box 搜索指定的两个地理位置（左上右下）形成的矩形范围中包含的酒店信息

```
GET /geo_point_case/_search
{
  "query":{
    "bool":{
      "must":[
        {"match_all":{}}
      ],
      "filter":{
        "geo_bounding_box": {
```

```
            "hotel_location": {
              "top_left": {
                "lat": 40.73,
                "lon": -74.1
              },
              "bottom_right": {
                "lat": 40.717,
                "lon": -73.99
              }
            }
          }
        }
      }
    }
```

- 检索数据- 情形2 geo_polygon 搜索指定的多个地理位置形成的多边形范围中包含的酒店信息

```
GET /geo_point_case/_search
{
  "query":{
    "bool":{
      "must":[
        {"match_all":{}}
      ],
      "filter":{
        "geo_polygon": {
          "hotel_location": {
            "points": [
              {
                "lat": 40.73,
                "lon": -74.1
              },
              {
                "lat": 40.83,
                "lon": -75.1
              },
              {
                "lat":50.56,
                "lon":-90.58
              }
            ]
          }
        }
      }
    }
  }
}
```

- 检索数据- 情形3 搜索指定位置10km范围内的酒店数据

```
GET /geo_point_case/_search
```

```
{
  "query":{
    "bool":{
      "must":[
        {"match_all":{}}
      ],
      "filter":{
        "geo_distance":{
          "distance":10,
          "hotel_location":{
            "lat": 40.73,
            "lon": -74.1
          }
        }
      }
    }
  }
}
```

- 检索数据- 情形4 搜索距离指定位置一定范围内有多少个酒店

  origin : 指定位置

  ranges : 范围
  unit : 范围使用单位

```
GET /geo_point_case/_search
{
  "size":0,
  "aggs":{
    "count_by_distinct":{
      "geo_distance":{
        "field":"hotel_location",
        "origin":{
          "lat":40,
          "lon":70
        },
        "ranges":[
          {"to":100},
          {"from":100, "to":300},
          {"from":300}
        ],
        "unit":"mi",
        "distance_type": "arc" // arc:最高经度,plane:最高效率,sloppy_arc:默认算法
      }
    }
  }
}
```

2.2.4.5 nested类型（嵌套类型）

- nested（嵌套）类型**是object数据类型的特殊版本**，可以在对象中嵌套对象或在字段中存储键值对（Key-ValuePair）
- 创建索引 nested_case

```
PUT /nested_case
{
  "mappings":{
    "properties":{
      "people":{
        "type":"nested"
      }
    }
  }
}
```

- 插入数据

```
PUT /nested_case/_doc/1
{
  "people":[
    {
      "first":"tommy",
      "last":"tang"
    },
    {
      "first":"sally",
      "last":"liu"
    }
  ]
}
```

- 查看映射

```
GET /nested_case/_mapping
```

映射如下

```
{
  "nested_case": {
    "mappings": {
      "properties": {
        "people": {
          "type": "nested",
          "properties": {
            "first": {
              "type": "text",
              "fields": {
                "keyword": {
                  "type": "keyword",
```

```
                    "ignore_above": 256
                  }
                }
              },
              "last": {
                "type": "text",
                "fields": {
                  "keyword": {
                    "type": "keyword",
                    "ignore_above": 256
                  }
                }
              }
            }
          }
        }
      }
    }
  }
```

- 检索数据

```
GET /nested_case/_search
{
  "query":{
    "nested":{
      "path":"people",
      "query":{
        "bool":{
          "must":[
            {"match":{"people.first":"tommy"}},
            {"match":{"people.last":"tang"}}
          ]
        }
      }
    }
  }
}
```

2.2.4.6 多字段

- 按照用户姓名进行搜索，又希望按照姓氏进行排列
- 创建索引

```
PUT /order
{
  "mappings":{
    "properties": {
      "order_id":{
```

```
          "type":"keyword"
        },
        "user_id":{
          "type":"keyword"
        },
        "user_name":{//多字段
          "type":"text",
          "fields":{
            "user_name_keyword":{
              "type":"keyword"
            }
          }
        },
        "hotel_id":{
          "type":"keyword"
        }
      }
    }
  }
```

- 批量写入数据

```
POST /_bulk
{"index":{"_index":"order", "_id":"001"}}
{"order_id":"1", "user_id":"x001", "user_name":"Michael Jordan", "hotel_id":"h0501"}
{"index":{"_index":"order","_id":"002"}}
{"order_id":"2","user_id":"x002","user_name":"Stephen Demon","hotel_id":"h0502"}
{"index":{"_index":"order","_id":"003"}}
{"order_id":"3","user_id":"x003","user_name":"Tim Jason","hotel_id":"h0503"}
{"index":{"_index":"order","_id":"004"}}
{"order_id":"4","user_id":"x004","user_name":"Richale Jordan","hotel_id":"h0504"}
{"index":{"_index":"order","_id":"005"}}
{"order_id":"5","user_id":"x005","user_name":"Paul Jordan","hotel_id":"h0505"}
```

- 检索user_name中有Jordan的数据，并根据user_name中last_name排序

```
GET /order/_search
{
  "query":{
    "match":{
      "user_name":"Jordan"
    }
  },
  "sort":{//根据user_name这个多字段下main的user_name_keyword升序排序
    "user_name.user_name_keyword":"asc"
  }
}

```

2.3 文档操作

- 创建索引 doc_case

```
PUT /doc_case
{
  "mappings":{
    "properties":{
      "name":{"type":"text"},
      "age":{"type":"integer"},
      "job":{"type":"keyword"}
    }
  }
}
```

2.3.1 单条写入

- 写入一条数据

```
POST /doc_case/_doc
{
  "name":"John",
  "age":25,
  "job":"Engineer"
}
```

2.3.2 批量写入

- 方式一：

```
POST /_bulk
{"index":{"_index":"doc_case","_id":"002"}}
{"name":"Sally", "age":23, "job":"Music Player"}
{"index":{"_index":"doc_case","_id":"003"}}
{"name":"Tommy", "age":28, "job":"developer"}
{"index":{"_index":"doc_case", "_id":"004"}}
{"name":"Jason", "age":21, "job":"student"}
```

- 方式二：

  基于json文件，curl命令批量写入

2.3.3 更新单条文档

- 方式一： update

```
POST /doc_case/_update/002
{
  "doc": {
    "age":25,
    "job":"孟智超"
  }

GET /doc_case/_search
```

- 方式2二：upsert 如果目标文档存在，则执行更新逻辑；**否则执行插入逻辑**

```
POST /doc_case/_update/003
{
  "doc":{ // 更新操作
    "name":"Jacky",
    "age":21,
    "job":"software developer"
  },
  "upsert":{ // 插入操作
    "name":"Jacky",
    "age":18,
    "job":"Jogging"
  }
}

GET /doc_case/_search
```

2.3.4 批量更新文档

- 与批量写入文档不同的是，批量更新文档必须在元数据中填写需要更新的文档_id

```
POST /_bulk
{"update":{"_index":"doc_case","_id":"002"}}
{"doc":{"name":"Sally Liu"}}
{"update":{"_index":"doc_case","_id":"003"}}
{"doc":{"name":"Jacky Wang"}}


GET /doc_case/_search
```

2.3.5 根据条件更新文档

- 更新文档的查询条件： name="Jacky Wang"

```
POST /doc_case/_update_by_query
{
  "query":{
    "match":{
      "name":"Jacky Wang"
    }
  },
  "script":{//更新脚本
    "lang":"painless",//默认就为painless
    "source":"ctx._source['job']='AI developer'"
  }
}
```

参考官网连接

> **https://www.elastic.co/guide/en/elasticsearch/painless/master/painless-update-context.html**

```
GET /doc_case/_search
{
  "query":{
    "match":{
      "name":"Jacky Wang"
    }
  }
}
```

2.3.6 删除单条文档

- 删除单条文档

```
DELETE /doc_case/_doc/002
```

2.3.7 批量删除文档

- 批量删除文档

```
POST /_bulk
{"delete":{"_index":"doc_case","_id":"6fUeaYMBostr4HR1R0BG"}}
{"delete":{"_index":"doc_case","_id":"003"}}
```

2.3.8 根据条件删除文档

- 根据条件删除文档

```
POST /doc_case/_delete_by_query
{
  "query":{
    "match":{
      "name":"Jason"
    }
  }
}

GET /doc_case/_search
```

3 es python api

- 安装库

> pip install elasticsearch

```python
from elasticsearch import Elasticsearch
from elasticsearch.helpers import bulk
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")
```

3.1 python连接

- 连接时需注意，如果没有将配置项修改，默认需要指定 `ssl_assert_fingerprint` ，和 `账号密码`

```python
# 1. 连接ES

CERT_FINGERPRINT = '9079dc2b753afa04ef601fb239f2d3b2a6239c9a986504df838e4cc64bc37796'
ELASTIC_PASSWORD = "sHZL+UMbav+xzdSzDw0Y"

# 方法一：需要添加用户名和密码
es = Elasticsearch("https://localhost:9200"
                   , ssl_assert_fingerprint=CERT_FINGERPRINT
                   , basic_auth=('elastic', ELASTIC_PASSWORD))

# #方法二：无需添加
# es = Elasticsearch("http://localhost:9200")

if es.ping():
    print('elasticsearch连接成功！！！')
else:
    print('连接失败！！！')
```

pandas读取数据用于后续插入

```python
# 2. 读取数据
df = pd.read_excel("./movie_plots_deduped_100.xlsx")
df.dropna(inplace=True) # 清理掉有null的数据，否则，es写入会报错
# 随机选择100条测试
df = df.sample(100, random_state=666)
print('df.columns, ',df.columns.tolist())

"""输出如下"""
df.columns, ['Release Year', 'Title', 'Origin/Ethnicity', 'Director', 'Cast', 'Genre', 'Wiki Page', 'Plot']
```

3.2 操作

3.2.1 创建索引

- 创建索引

```python
# ① 定义mappings（映射）
mappings = {
        "properties": {
            "title": {"type": "text"},
            "ethnicity": {"type": "text"},
            "director": {"type": "text"},
            "cast": {"type": "text"},
            "genre": {"type": "text"},
            "plot": {"type": "text"},
            "year": {"type": "integer"},
            "wiki_page": {"type": "keyword"}
    }
}

# 判断是否存在movies索引，如果存在则删除
if es.indices.exists(index="movies"):
    es.indices.delete(index="movies")
# 创建movies索引
es.indices.create(index="movies", mappings=mappings)
```

3.2.2 插入数据

- 方式一：单挑插入

```python
# 方式1：.index() 一次插入一条数据
start = time.time() # 开始时间

for i, row in df.iterrows():
    doc = {
        "title": row["Title"],
```

```
        "ethnicity": row["Origin/Ethnicity"],
        "director": row["Director"],
        "cast": row["Cast"],
        "genre": row["Genre"],
        "plot": row["Plot"],
        "year": row["Release Year"],
        "wiki_page": row["Wiki Page"]
    }

    es.index(index="movies", id=i, document=doc) # 单条插入数据

print("总耗时: ", time.time() - start)

"""输出"""
5.385900974273682
```

- 方式二：多条插入

```
# 方式2: .bulk() 同时添加多条数据

# 先清空索引，重新插入数据
if es.indices.exists(index="movies"):
    es.indices.delete(index="movies")
es.indices.create(index="movies", mappings=mappings)




start = time.time() # 开始时间

bulk_data = []

for i, row in df.iterrows():
    bulk_data.append({
        "_index": "movies",
        "_id": i,
        "_source":{
            "title": row["Title"],
            "ethnicity": row["Origin/Ethnicity"],
            "director": row["Director"],
            "cast": row["Cast"],
            "genre": row["Genre"],
            "plot": row["Plot"],
            "year": row["Release Year"],
            "wiki_page": row["Wiki Page"]
        }
    })

# 批量写入es
from elasticsearch.helpers import bulk

```

```
bulk(es, bulk_data)

print("总耗时: ", time.time() - start)

```

3.2.3 查看索引中的文档数量

- 查看索引中的文档数量

```
es.indices.refresh(index="movies") # 刷新

es.cat.count(index="movies", format="json") # 统计，以json格式显示

"""输出"""
ListApiResponse([{'epoch': '1728207119', 'timestamp': '09:31:59', 'count': '100'}])
```

3.2.4 查询所有

- 查询所有

```
body = {
    "match_all": {}
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.5 根据term查询

- term主要用于 `精确匹配` 哪些值，比如数字，日期，布尔值或 not_analyzed 的字符串(`未经切词的文本数据类型`)

```
body = {
    "term": {
        "wiki_page": 'https://en.wikipedia.org/wiki/Anna_Karenina_(1915_film)'
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.6 根据 terms 查询

- terms 跟 term 有点类似，但 terms 允许指定多个匹配条件。 如果某个字段指定了多个值，那么文档需要一起去做匹配

```
body = {
    "terms": {
        "year": [2004, 2002]
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.7 range

- 按照指定范围查找一批数据:

```
body = {
    "range":{
        "year": {
            "gt": 2000
        }
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.8 exists / missing

- 查找文档中是否包含指定字段或没有某个字段，类似于SQL语句中的IS_NULL条件

```
body = {
    "exists": {
        "field": "cast"
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.9 bool过滤

- 合并多个过滤条件查询结果的布尔逻辑
    1. `must` :多个查询条件的完全匹配,相当于 and。

2. `must_not` :多个查询条件的相反匹配，相当于 not。
3. `should` :至少有一个查询条件匹配, 相当于 or。

```
body = {
    "bool": {
        "must": {
            "terms": {"year": [1914,2004,2002,1984,2017,2005]}
        },
        "filter":{
            "match":{"genre":"horror"}
        }
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.10 multi_match

- match查询的基础上同时搜索多个字段，在多个字段中同时查一个

```
body = {
    "multi_match": {
        "query": "romantic",
        "fields": ["genre","plot"]
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.11 regexp 查询

- regexp 类似模糊查询

```
body = {
    "regexp": {
        "director": "john*"
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.12 prefix 查询

- 以什么字符开头

```
body = {
    "prefix": {
        "director": "john"
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.13 phrase match 短语匹配

- 寻找临近的几个单词

```
body = {
    "match_phrase": {
        "plot": "quickly"
    }
}

result = es.search(index="movies", query=body)

print(result)
```

3.2.14 根据id删除数据

- 根据id删除数据

```
es.delete(index="movies", id='1')
```

3.2.15 delete_by_query

- 删除满足条件的所有数据，查询条件必须符合DLS格式

```
body = {
    "query":{
        "match": {
            "genre": "crime"
        }
    }
}

result =es.delete_by_query(index="movies", body=body)

print(result)
```

4 案例

4.1 python+BERT+es(英文)

4.1.1 简介

- 使用 `sentence-transformers` 将英文qa对 embedding,计算文本相似性，召回相对应结果
- `sentence-transformers` 是一个Python库，它提供了一些预训练模型，这些模型可以用于将句子转换成固定长度的向量，这些向量可以用于各种下游任务，如句子相似度计算、句子分类等。`sentence-transformers` 和 `transformers` 库在处理句子嵌入时采用了不同的方法。
    1. **`sentence-transformers` 库**：
        - 它专门用于生成句子嵌入，通常在预训练模型的基础上进行了额外的训练或微调，以优化句子级别的表示。
        - 它通常会使用一些特定的技术，如平均词嵌入（mean pooling）或更复杂的池化策略，以生成句子的嵌入向量。
    2. **`transformers` 库**：
        - 它提供了BERT和其他模型的原始实现，可以用于各种NLP任务，包括句子嵌入。
        - 在上面的示例中，我们直接使用了BERT模型的最后一层的[CLS] token的输出作为句子嵌入。这种方法简单直接，但可能不如 `sentence-transformers` 库中的模型那样经过优化。

4.1.2 代码

- 导库

```python
from sentence_transformers import SentenceTransformer
from elasticsearch import Elasticsearch

import pandas as pd
import warnings
warnings.filterwarnings('ignore')
```

- 加载预训练模型

```python
# 加载预训练模型
model = SentenceTransformer("bert-base-nli-mean-tokens"
                            ,device='cuda'
                           )
```

- 读数

```python
# 读数
df = pd.read_excel('./data/english_qa_pairs_10.xlsx')
df
```

	question	answer
0	Is Python a programming language?	Yes, Python is a widely-used high-level progra...
1	Is the Earth round?	Yes, the Earth is an oblate spheroid, meaning ...
2	What is the chemical formula for water?	The chemical formula for water is H2O.
3	How many chambers does the human heart have?	The human heart has four chambers: two atria a...
4	What is the speed of light?	The speed of light in a vacuum is approximatel...
5	What is quantum mechanics?	Quantum mechanics is a branch of physics that ...
6	What is the tallest mountain in the world?	The tallest mountain in the world is Mount Eve...
7	What is artificial intelligence?	Artificial intelligence is a branch of compute...
8	How many planets are there in the solar system?	There are eight planets in the solar system: M...
9	What is DNA?	DNA stands for deoxyribonucleic acid, which is...

- 构建qa对

```python
questions=[]
qa_pairs = []
for k,v in df.values:
    questions.append(k)
    qa_pairs.append({"question":k,"answer":v})
qa_pairs
```

qa对

```
[{'question': 'Is Python a programming language?',
  'answer': 'Yes, Python is a widely-used high-level programming language.'},
 {'question': 'Is the Earth round?',
  'answer': 'Yes, the Earth is an oblate spheroid, meaning it is slightly flattened at the poles and bulging at the equator.'},
 {'question': 'What is the chemical formula for water?',
  'answer': 'The chemical formula for water is H2O.'},
 {'question': 'How many chambers does the human heart have?',
  'answer': 'The human heart has four chambers: two atria and two ventricles.'},
 {'question': 'What is the speed of light?',
  'answer': 'The speed of light in a vacuum is approximately 299,792 kilometers per second.'},
 {'question': 'What is quantum mechanics?',
  'answer': 'Quantum mechanics is a branch of physics that studies the behavior and interactions of matter and energy on very small scales.'},
 {'question': 'What is the tallest mountain in the world?',
```

```
  'answer': 'The tallest mountain in the world is Mount Everest, located in the Himalayas.'},
 {'question': 'What is artificial intelligence?',
  'answer': 'Artificial intelligence is a branch of computer science that aims to understand the essence of intelligence and produce new intelligent machines that respond in a manner similar to human intelligence.'},
 {'question': 'How many planets are there in the solar system?',
  'answer': 'There are eight planets in the solar system: Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, and Neptune.'},
 {'question': 'What is DNA?',
  'answer': 'DNA stands for deoxyribonucleic acid, which is the molecule that stores genetic information in living organisms.'}]
```

- 测试模型是否可用

```
questions_embeddings = model.encode(questions)
questions_embeddings

for q,q_emb in zip(questions,questions_embeddings):
    print(f"{q} ------ embeddomg shape:{q_emb.shape}")


"""输出结果"""
Is Python a programming language? ------ embeddomg shape:(768,)
Is the Earth round? ------ embeddomg shape:(768,)
What is the chemical formula for water? ------ embeddomg shape:(768,)
How many chambers does the human heart have? ------ embeddomg shape:(768,)
What is the speed of light? ------ embeddomg shape:(768,)
What is quantum mechanics? ------ embeddomg shape:(768,)
What is the tallest mountain in the world? ------ embeddomg shape:(768,)
What is artificial intelligence? ------ embeddomg shape:(768,)
How many planets are there in the solar system? ------ embeddomg shape:(768,)
What is DNA? ------ embeddomg shape:(768,)
```

- 创建es对象

```
# 创建es对象
es = Elasticsearch("https://localhost:9200"
,ssl_assert_fingerprint='3d406b2951d84ddc52d27dddf6b7f0c7c999fd0126048d0757d34294bcde7d8d'
                   ,basic_auth=('elastic','-o8A2JNGOlsoEF285ZBg'))
es.ping()

"""输出结果"""
True
```

- 创建索引

```python
# 创建索引
def create_index(query):
    # 如果索引存在则删除，否则忽略404报错
    es.indices.delete(index='english_index',ignore=404)
    # 创建索引
    es.indices.create(index='english_index',body=query)

# 索引结构
query = {
    "mappings":{
        "properties":{
            "question":{
                "type":"text"
            },
            "answer":{
                "type":'text'
            },
            "embedding":{
                "type":'dense_vector',
                "dims":768
            }
        }
    }
}
create_index(query)
```

- 将数据写入es

```python
def input_data_to_es(qa_pairs):
    """单独一条插入，实际不会用"""
    for qa in qa_pairs:#遍历语句对
        question = qa['question'] # 问题
        answer = qa['answer'] # 回答
        question_embedding = model.encode(question).tolist()
        # 构建成与索引的结果一致
        query = {
            "question" : question,
            "answer":answer,
            "embedding":question_embedding
        }
        # 写入数据
        es.index(index='english_index',body=query)

input_data_to_es(qa_pairs)
```

- 定义检索-召回方法，计算文本相似性，返回结果

```python
# 定义检索-召回方法，计算文本相似性，返回结果
def query_qa(question:str,topn:int=2):
    # 对用户输入question进行encoding
```

```python
    question_emb = model.encode(question).tolist()
    # ES召回+根据embedding计算文本相似性score
    res = es.search(
                    index='english_index' # 指定索引为english_index
                   ,body={
                        "size":topn, # 返回结果数量
                        "_source":["question","answer"], # 在返回最相似的topn，必须包含 question 和answer
                        "query":{# 定义查询逻辑
                            "script_score":{ #  用于计算脚本分数，它结合了基本查询（match）和自定义脚本
                                "query":{
                                    "match":{
                                        "question":question # 查询用于找到包含用户问题的候选回答。
                                    }
                                },
                                "script":{ # 定义了一个脚本，使用余弦相似度（cosineSimilarity）计算查询向量和文档嵌入向量之间的相似度，并将结果加1.0（通常是为了将分数转换为非负值）
                                    "source":"""(cosineSimilarity(params.query_vector,"embedding") + 1.0)""",
                                    "params":{"query_vector":question_emb}
                                }
                            }
                        }
                   })
    values = res['hits']['hits']# 从 ES 返回的结果中提取出实际的命中结果。
    filtered_results = list() # 初始化一个空列表 filtered_results，用于存储过滤后的结果
    for item in values:
        temp = {
            "question":item['_source']['question'],
            "answer":item['_source']['answer'],
            "score":item['_score'] - 1.0
        }
        filtered_results.append(temp)

    # 函数返回 filtered_results 列表，包含了最相似的问题、答案和分数。
    return filtered_results
```

- 测试

```python
# 测试
input_question = "python is a programming language?"
query_results = query_qa(question=input_question
                        ,topn=3)
query_results

"""输出结果"""
```

```
[{'question': 'Is Python a programming language?',
  'answer': 'Yes, Python is a widely-used high-level programming language.',
  'score': 0.9774565},
 {'question': 'What is artificial intelligence?',
  'answer': 'Artificial intelligence is a branch of computer science that aims to understand the essence of intelligence and produce new intelligent machines that respond in a manner similar to human intelligence.',
  'score': 0.5979325},
 {'question': 'What is DNA?',
  'answer': 'DNA stands for deoxyribonucleic acid, which is the molecule that stores genetic information in living organisms.',
  'score': 0.4900960000000001}]
```

---

pip install -U bert-serving-server bert-serving-client

bert-serving-start -model_dir C:\Users\孟智超\Downloads\es_chinese\chinese_macbert_base -ckpt_name chinese_macbert_base.ckpt -config_name macbert_base_config.json -max_seq_len 64 -port 5555 -port_out 5556 -num_work 8 -max_batch_size 32